

Instructions for experts

Welcome to Project Silver Knowledge Check

This is a basic check to ensure you understand the project requirements and what is and isn't allowed for submissions. You must score **100%** on the quiz to pass and continue in the project.

This is **open note and open book** — feel free to refer back to the instructions as you work through the questions. You have **unlimited attempts**, so take your time and make sure you genuinely understand the material before moving on. The pipeline rules are strict, and a few minutes spent here will save you wasted submission attempts later.

01) Overview
Module 2 of 12

Silver produces SWE-bench-style coding tasks. Each task is a real bug or feature request grounded in a private repository, with a machine-verifiable test harness and a reference solution. These tasks are used to evaluate how well AI agents can write code in realistic settings.

The end-to-end flow has three phases:

1. Repository submission - you upload a private codebase and a Dockerfile that builds it.
2. Task authoring - you write task instructions, tests, and a reference solution against a specific commit of that repo.
3. Validation pipeline - the platform runs your task through automated checks, including live agent runs, and tells you whether it passes.

You iterate on phases 2 and 3 until the pipeline approves your task. Most tasks take 2-4 submission attempts; the feedback is specific enough to know exactly what to fix each round.

Assessment context: If you're going through the silver assessment, phase 1 is already done for you - ops curates a practice repo and you author one task against it. You have 5 attempts with a 15-minute cooldown between verdict failures. Pass all five pipeline stages and you're granted the expert role.

02) Step 1: Submit a repository

Module 3 of 12

Already done for the assessment. The practice repo is curated and uploaded by ops — you don't go through this step. Click Download repo (.zip) on the assessment page if you want to author offline. The rules below apply once you start contributing your own repos post-pass.

Go to Repositories > Upload repository and upload a .zip of your private codebase. The zip must include the .git/ directory - the platform needs full commit history to reset to specific commits during task execution.

REPOSITORIES MUST BE PRIVATE ON GITHUB

Submit only repositories that are private on GitHub (or not on GitHub at all). The platform automatically probes public GitHub for your repo and rejects any match - including forks, mirrors, and renamed copies. The same applies to other public hosts (GitLab, Bitbucket, public Gitea instances, etc.): if your code is reachable publicly anywhere on the internet, don't submit it.

Submitting a public or open-source repository will be flagged, will result in rejection, and may result in removal from the project.

QUALITY MATTERS

We're looking for real, production-grade codebases from actual companies or genuine multi-author projects - not homework, hackathon submissions, personal toy projects, side projects, tutorials, course assignments, or boilerplate starters. The repo should reflect work that was done with intent over time.

REPOSITORY PAYOUT REQUIRES 5+ APPROVED TASKS

The \$300 per approved repository bonus is only paid out when the repo has at least 5 approved tasks authored against it. A repo on its own - without enough tasks landing on it - doesn't qualify for the repository payout. Per-task payouts (\$75 each) are still paid independently for every approved task.

03) Step 2: Write a Dockerfile Module 4 of 12

Already done for the assessment. The practice repo's base image is built by ops. You'll write a per-task Dockerfile in step 3 that uses it as the FROM base - the rules in this section apply there too.

Every repository needs a Dockerfile that builds the project and installs its dependencies. Upload it alongside your repo, or replace it later from the Repositories page.

The build system strips the top-level folder from your zip and places your Dockerfile at the root, so always use COPY . . - not COPY my-repo-name/ ..

The platform runs static checks before the build is even attempted. A single rule violation fails the image-build stage with the rule name in the error message.

SIZE BUDGET:

- <= 20 KB total Dockerfile file size
- <= 200 lines including blank lines and comments

- `<= 30 RUN` instructions

FROM DIRECTIVE:

- Pin the base image (e.g., `FROM node:20`)
- Per-task Dockerfiles must use the exact approved repo image reference
- `FROM scratch` is allowed without a tag

PINNED PACKAGES:

- `pip`: every package needs `==X.Y.Z`
- `npm/yarn/pnpm`: every explicit package needs `@X.Y.Z`
- `apt-get install`: must include `--no-install-recommends`

NETWORK INSTALLS:

- `curl` | `sh` / `wget` | `sh` need a versioned URL
- No `ADD http(s)://`

FORBIDDEN IN RUN:

- `--privileged` (never)
- `eval`
- Dynamic command construction
- Base64-like literals `>= 200` chars

04) Step 3: Author a task

Module 5 of 12

Step 3: Author a task

Once your repo is approved (or, for assessment candidates, once you click Start attempt), you have two ways to work:

OPTION A: Author on the platform

Click New task next to your repo (or Start attempt on the assessment page). Pick the base commit; you land in an in-browser editor with pre-filled template files for your language. Browse the repo source in a separate tab while you write. Hit Submit directly from the editor when ready.

OPTION B: Author locally, import when done

Click Template next to your repo to download a pre-filled zip for a specific commit. Edit in your own editor, run tests locally, then click Import to upload. The platform creates a draft from your files and you submit from there.

Starting from the downloaded template is strongly recommended over building the file structure from scratch — it ensures the right directory layout, gives you a working test harness and parser for your language, and pre-fills boilerplate like the Dockerfile and config.json so you can focus on the actual task content.

PICKING A BASE COMMIT

The base commit is the "broken" state the agent starts from. Choose the commit before the fix you have in mind. The agent will try to fix whatever your task describes, starting from this exact commit.

THE EDITOR (PLATFORM AUTHORIZING)

The in-browser editor has three tabs:

- Task workspace — where you edit the task files. This is where you spend most of your time.
- Repo files — a read-only browser of the repository source. Use this to reference code while writing instructions.

- Validation — shows pipeline progress and per-stage results after you submit.

WORKSPACE SIZE LIMITS

The platform enforces caps as you edit: ≤ 64 files, ≤ 256 KB per file, ≤ 768 KB total. Zip imports add their own: ≤ 2 MB zip, ≤ 512 KB per file, and the zip may contain only the files listed below.

THE FILES YOU EDIT

my-task/

- ├─ [instruction.md](#) What the agent reads
- ├─ reference_ [plan.md](#) Your root-cause + test-plan notes
- ├─ task.toml Metadata, timeouts, resource caps
- ├─ environment/
 - | └─ Dockerfile FROM
- ├─ solution/
 - | └─ [solve.sh](#) Unified diff wrapped in git apply
- └─ tests/
 - ├─ [test.sh](#) Harness entrypoint
 - ├─ run_ [script.sh](#) Language test runner
 - ├─ [parser.py](#) Parses runner stdout → JSON

└─ config.json base_commit, fail_to_pass, pass_to_pass, test_patch

[instruction.md](#) – WHAT THE AGENT SEES

Describe the bug or feature clearly. Include the specific file(s) and function(s) involved, expected vs. actual behavior with concrete inputs and outputs, and any constraints.

Describe what needs to happen, not how to implement it – no pseudocode, no prescribed variable names.

reference_[plan.md](#) – YOUR ROOT-CAUSE + TEST PLAN

Internal-facing companion to [instruction.md](#). Write a short root-cause analysis, a sketch of the intended fix, and what the tests are meant to cover. Reviewers read this when a task lands in human review – treat it as the "why this is a fair, hard task" document.

tests/config.json – TEST CONTRACT

Required keys: base_commit (7–40 hex chars, must exist in the repo), fail_to_pass (non-empty array of test names that fail at base_commit and pass after the fix), pass_to_pass (tests that pass before and after), test_patch (unified diff that adds the test files at base_commit if they aren't in the repo already), and selected_test_files_to_run. Test names must exactly match what your runner outputs.

solution/[solve.sh](#) – REFERENCE SOLUTION

A bash script that applies a unified-diff patch via git apply. The seed template gives you the wrapper; paste your real diff between the heredoc markers. The patch must apply cleanly at base_commit and must make every fail_to_pass test pass.

environment/Dockerfile – PER-TASK DOCKER ENVIRONMENT

Must use the approved repo image as its FROM base. The template is usually close to ready — tweak it if your task needs additional system packages or a different working directory. All Dockerfile rules from step 2 apply here too.

tests/run_[script.sh](#) — RUNS YOUR TEST SUITE

Pre-filled for your language (e.g. yarn test, cargo test, pytest, mvn test, go test). Adjust to match your project's actual test invocation.

tests/[parser.py](#) — PARSES TEST OUTPUT

Maps runner stdout/stderr to a list of {name, status} records. Handles Jest, pytest, cargo, Maven, and Go formats. Only modify if your runner emits something unusual.

task.toml — TASK METADATA

Set author_name, author_email, difficulty (one of easy, medium, hard), and category. Default timeouts and resource caps are fine for most tasks.

tests/[test.sh](#) — TEST HARNESS ORCHESTRATION

Main entrypoint the platform calls. The seed version applies test_patch, runs run_[script.sh](#), pipes through [parser.py](#), checks every test in fail_to_pass ∪ pass_to_pass passed, and writes 1/0 to /logs/verifier/reward.txt. You almost never modify it.

05) Step 4: Test locally with Harbor
Module 6 of 12

Step 4: Test locally with Harbor

Silver runs on the open-source Harbor framework. The same test.sh contract Harbor uses is what the server null/oracle stage runs — so two local commands reproduce that stage exactly. Every assessment attempt counts; run these locally first.

1. Install Harbor and Docker

Install the CLI and make sure Docker is running locally:

```
uv tool install harbor # recommended
```

or pip install harbor

1. Lay out your task

On the assessment page (or repo page for your own repos) click Download repo (.zip) — you'll get a signed URL that's valid for 15 minutes. Extract the repo, then drop your task scaffold (instruction.md, environment/Dockerfile, solution/solve.sh, tests/, task.toml) in a sibling directory. Confirm your config.json's basecommit *exists in the extracted repo before going further*.

1. Run the null check

```
harbor run -p ./my-task -a nop
```

The nop agent does nothing — it just runs tests/test.sh with no solution applied. Every test in failtopass must FAIL. /logs/verifier/reward.txt should contain 0. If it's 1, the bug is already fixed at your basecommit or your tests aren't actually exercising the bug.

1. Run the oracle check

```
harbor run -p ./my-task -a oracle
```

The oracle agent applies solution/solve.sh first, then runs the tests. Every test in failtopass $\cup$ passtopass must PASS. reward.txt should contain 1.

If oracle fails: either git apply errored on the patch (check the patch's context lines match base_commit), the patch applied but tests still fail (the fix is incomplete), or the test names in config.json don't match what parser.py emits.

1. Inspect logs

```
harbor view ./jobs
```

Opens an interactive viewer with stdout, stderr, parser output, and reward file for each run. Use this when oracle fails to see the exact runner output the parser saw.

Harbor only covers the null/oracle stage. The server pipeline also runs Dockerfile static checks (run them mentally against step 2's rules), the similarity check (server-only, requires the corpus), the rubric LLM (server-only), and the easiness probe (server-only). Passing both Harbor commands locally is necessary but not sufficient — it just rules out the failure mode that would have wasted an attempt.

06) What makes a good task

Module 7 of 12

What makes a good task

A good task is a real problem — something that actually happened in your codebase (or plausibly could). The best tasks come from real bugs, real feature requests, or real refactoring challenges. They should be hard for an AI agent to solve, but fair.

WHAT TO LOOK FOR IN YOUR CODEBASE

Bugs with non-obvious root causes

The symptom appears in one place but the fix is in another. The agent needs to trace through multiple files or understand a subtle interaction to find the real issue.

Cross-cutting changes

Tasks that touch multiple files or modules are harder than single-function fixes. Think: a type change that ripples through several call sites, or a config change that affects multiple subsystems.

Edge cases and subtle invariants

Off-by-one errors, race conditions, encoding issues, boundary conditions in business logic. These require careful reasoning, not just pattern matching.

Non-trivial feature additions

Small features that require understanding the existing architecture to integrate correctly. The agent needs to figure out the right place to add code, not just write new code in isolation.

TASKS NEED TO LAND IN THE DIFFICULTY SWEET SPOT

The difficulty probe runs your task against a state-of-the-art AI agent 10 times. To be accepted, the agent must solve it in 1–4 of those 10 runs:

- 0/10 — rejected as unverifiable. Either the task is unsolvable as specified, or the test harness has a bug preventing any agent from passing.
- 1–4/10 — accepted. The task is hard but solvable with careful reasoning.
- 5+/10 — rejected as too easy. The agent can solve it on most attempts, which means it isn't exercising the kind of reasoning we're trying to evaluate.

This is a high bar — tasks that feel moderately difficult to a human are often trivially solvable by an agent that can search and read the entire codebase instantly.

Difficulty should come from the problem itself

A hard task has a non-obvious root cause, requires understanding a complex interaction, or involves a subtle edge case. A task that's hard because the instructions are vague or the tests are broken is not a good task — the rubric will catch this and reject it.

Think about what the agent can do easily

Agents are extremely good at: searching code, reading documentation, following clear instructions, making straightforward fixes. Your task needs to require something more — reasoning about runtime behavior, understanding implicit constraints, or connecting information across distant parts of the codebase.

WRITING GOOD INSTRUCTIONS

Describe outcomes, not implementations

Say "the `/api/users` endpoint should return 404 when the user is deleted" instead of "add an if-check in `UserController.getUser` that returns 404." The agent needs to figure out the how.

Be concrete about what correct looks like

Include specific inputs and expected outputs where possible. "The function should handle edge cases" is vague. "Calling `processOrder({ items: [] })` should throw an `EmptyCartError`" is testable.

Keep formatting clean

Avoid walls of text. Use short paragraphs with line breaks. Markdown headers are fine for longer tasks. Don't dump config files or stack traces into the instructions unless they're genuinely necessary context.

Don't leak the solution

Don't name the exact variable, function signature, or algorithm the solution should use. Don't include pseudocode. The task should be solvable from the description of the bug and the expected behavior alone.

WRITING GOOD TESTS

Test behavior, not implementation

Good tests run the code and check outputs. Bad tests grep the source for variable names, scan for specific import statements, or regex-match function signatures. If a correct but differently-structured solution would fail your test, the test is brittle and the pipeline will flag it.

Tests must actually execute the code

Every test in `fail_to_pass` must genuinely call the code path the task is about and assert on the result. A test that just checks if a file exists or if a string appears in the source isn't verifying behavior.

Get the test names right

The names in `fail_to_pass` and `pass_to_pass` must exactly match what your test runner outputs. Run your tests locally and copy the names from the output. Mismatched names are a common cause of validation failures.

Test isolation matters

Tests should not depend on execution order, network access, or mutable shared state. The container is fresh for every run, but tests within a run share a process.

07) The rubric Module 8 of 12

The rubric

The rubric stage is the most common source of verdict failures. An LLM grades your task against the 11 criteria below. The verdict must be `Accept` or `Strong Accept` – anything else (`Uncertain`, `Reject`, `Strong Reject`) fails this stage. The list is kept in sync with the LLM prompt at `app/projects/silver/lib/pipeline/rubric-review.ts`.

01 — Verifiable

Tests are deterministic and produce a clear pass/fail signal. No subjective judgment in the pass condition.

02 — Well-specified

A competent engineer could reproduce the intended outcome from the instruction alone, without insider knowledge of your repo or your fix.

03 — Solvable

Achievable within the repo's scope and a reasonable time budget. Not days of work; not requiring multiple unrelated fixes.

04 — Genuinely difficult

The challenge is intellectual, not formatting busywork. The agent has to reason about runtime behavior, find a non-obvious cause, or connect distant parts of the codebase.

05 — Behavioral verification

Tests execute the code and assert on its actual behavior. They never grep source for variable names, scan imports, or regex-match function signatures as a stand-in for correctness.

06 — Outcome-verified

The instruction describes what should be true after the fix, not how to implement it. No pseudocode, no dictated function names, no step-by-step implementation guidance.

07 — Test-instruction alignment

fail_to_pass tests fail at base_commit and pass after [solve.sh](#). pass_to_pass tests pass throughout. Tests cover everything the instruction asks for, and nothing more.

08 — Instruction quality

Concise, human-written prose. Short paragraphs. Headers if it helps. Not a wall of stack traces; not a copy-pasted config dump as the body of the description.

09 — Fair

Self-contained. The agent shouldn't need information not present in the repo or the instruction itself. No insider tribal knowledge.

10 — Anti-cheat robustness

Tests can't be passed by hardcoding outputs, monkey-patching the system under test, or wrapping it in a fake. A solution that doesn't actually fix the bug should fail.

11 — Deterministic & reproducible

Repeated runs produce the same result. No external services that change between runs. No time-dependent assertions. No nondeterministic ordering.

08) The validation pipeline Module 9 of 12

The validation pipeline

When you click Submit on a draft, it enters the pipeline. Each stage is pass/fail — if any stage fails, the pipeline stops, the draft unlocks, and you can revise and resubmit. The thresholds below are the actual numbers the server enforces.

1. Similarity check

$\text{cosine} \geq 0.75$ OR $\text{Levenshtein} \geq 0.70$

Your instruction is compared against existing tasks. Patch-hash dedup also runs against the exact bytes of solve.sh.

1. Rubric review

Accept or Strong Accept only

An LLM grades your task against the 11 rubric criteria above. Per-criterion notes appear in the validation tab when this stage fails.

1. Image build

static checks + build success

Static rules from step 2 run first. If they pass, your per-task Dockerfile is built in the cloud.

1. Null & oracle

null FAILs, oracle PASSes

Two parallel runs. Null: no solution applied — failtopass tests must FAIL. Oracle: solve.sh applied — failtopass $\cup$ passtopass tests must PASS.

1. Easiness probe

agent must NOT solve trivially

A model attempts your task once inside the built container with no hints. If it solves the task on this single attempt, the task is rejected as too easy.

Production tasks run two more stages after the easiness probe: a difficulty probe (10 Opus runs — your task must land in the 1–4 / 10 sweet spot) and an LLM trajectory review of the failed attempts. Assessments stop after stage 5 — passing all five flips your account to passed and grants the expert role.

09) Common pitfalls

Module 10 of 12

Common pitfalls

Each pitfall below is tagged with the stage that catches it, so you know where in the pipeline it'll surface and what to read next.

STAGE 4 · NULL — Tests pass without any solution

Your base commit already has the fix, or the test doesn't actually exercise the bug. Choose an earlier commit where the bug is present, or write a more targeted test.

STAGE 4 · ORACLE — Tests fail even with the solution

Three usual causes: (1) the patch doesn't apply cleanly at base_commit — check the context lines; (2) the patch applies but is incomplete; (3) test names in config.json don't match [parser.py](#) output. Run `harbor run -a oracle` locally and inspect parser output.

STAGE 5 · EASINESS — The agent solved your task on its single attempt

The problem itself isn't hard enough — and making the instructions vaguer won't help, the model will still see the bug clearly. You need a problem with a non-obvious root cause, a subtle interaction between components, or a tricky edge case.

STAGE 6 · DIFFICULTY — More than 4 of 10 difficulty-probe runs solved it

Same root cause as the easiness probe — the bug is too easy for a frontier model. The difficulty probe is stricter: 5+ of 10 runs solving means the task isn't exercising the kind

of reasoning we're trying to evaluate. Make the bug subtler or constrain the test more aggressively.

STAGE 6 · DIFFICULTY — 0 of 10 difficulty-probe runs solved it

The task is unverifiable — either the test harness is broken (no agent can pass even with the correct fix) or the task as specified has no solution. Run the oracle locally to confirm your patch passes, then verify the harness actually gates on the right tests.

STAGE 2 · RUBRIC — Tests check implementation details (brittle)

Your tests are checking implementation details (variable names, specific strings in source, import statements) rather than running code and checking outputs. Rewrite to test behavior. Fails criterion 5.

STAGE 2 · RUBRIC — Instructions are over-prescriptive

Your instructions include pseudocode, specific variable names, or step-by-step implementation guidance. Remove these and describe only the expected behavior and constraints. Fails criterion 6.

STAGE 1 · SIMILARITY — Cosine ≥ 0.75 or Levenshtein ≥ 0.70 against an existing task

Your instruction is too close to one already in the system. Rewrite from scratch — paraphrasing rarely moves the embedding far enough. Pick a different bug or feature.

STAGE 3 · IMAGE BUILD — Image build fails

Check the build log in the validation tab. Common causes: your per-task Dockerfile's FROM image doesn't match the approved repo image, a dependency install step fails, or a system package is missing.

STAGE 3 · IMAGE BUILD — Dockerfile rejected for unpinned packages

Pin every package: `pip install pytest==8.2.2`, `npm install jest@29.7.0`. Unversioned installs are rejected because they make builds non-reproducible.

STAGE 3 · IMAGE BUILD — apt-get without --no-install-recommends

Add the flag to every apt-get install line, even short ones. The static checker rejects any line missing it.

STAGE 3 · IMAGE BUILD — COPY destination outside allowlist

COPY only to `/app`, `/workspace`, `/tmp`, `/opt`, `/usr/local`, or `/home`. You get up to 3 exceptions outside that set; more than 3 fails the check.

PRE-PIPELINE — Dockerfile FROM doesn't match approved repo image

Use the exact approved image reference shown on the assessment / repo page. Copy it verbatim into your FROM line.

PRE-PIPELINE — No changes since last submission

The platform hashes content to prevent identical resubmissions. Edit something material — a fix that changes the patch bytes, an instruction rewrite, an updated test name.

10) Practical workflow
Module 11 of 12

Practical workflow

Here's the process most experienced task authors follow. Step 4 in particular saves attempts — running Harbor locally catches the most common stage-4 failures before they cost you a verdict.

1. Pick a real bug or feature from your repo's history

Look at your commit log for changes that were non-trivial to implement, touched multiple files, or fixed a subtle issue. The commit before the fix is your base commit.

1. Download the template

On the Create tasks page (or the assessment page), click Template or Download repo (.zip) and pick your commit. This gives you a zip with the right directory structure and language-specific boilerplate already filled in. Open it in your editor of choice.

1. Write the instruction first

Before touching any other file, write what the agent should see. If you can't describe the task without leaking the solution, it might not be a good candidate.

1. Run Harbor locally until null fails and oracle passes

`harbor run -p ./my-task -a nop must report failtopass FAILing.` `harbor run -p ./my-task -a oracle must report all required tests PASSing.` This rules out stage-4 failures, which otherwise eat an attempt for a problem you could have caught in 90 seconds locally.

1. Re-read your instruction against the 11 rubric criteria

Specifically: would a fresh engineer reproduce the outcome without insider context? Is anything an implementation hint? Is anything that looks like a stack trace actually load-bearing?

1. Import or edit on the platform

Click Import to upload your local zip, or click New task / Start attempt and edit directly in the browser. Either way, you end up with a draft you can submit.

1. Submit and read the validation tab in full

The pipeline feedback is specific and actionable — it's faster to iterate on real feedback than to guess what the pipeline wants. When the rubric stage fails, per-criterion notes are included.

1. Iterate offline before resubmitting

Each draft tracks all previous submissions in the past-runs expander. Don't resubmit with the same issue. For assessments, the 15-min cooldown is there for a reason — spend it actually fixing the cited issue.

1. If you hit the 20-task cap, delete a non-approved draft

Each repo allows up to 20 of your tasks — the goal is 20 approved. Only Approved drafts are protected from deletion (they're paid and can't be reclaimed). Anything else — Needs Review, Rejected, Validation Failed, or never submitted — is deletable from the Create tasks page and frees a slot immediately.